Day 1

글을 작성하세요. 블록을 추가하려면 '/'를 입력하세요

목차

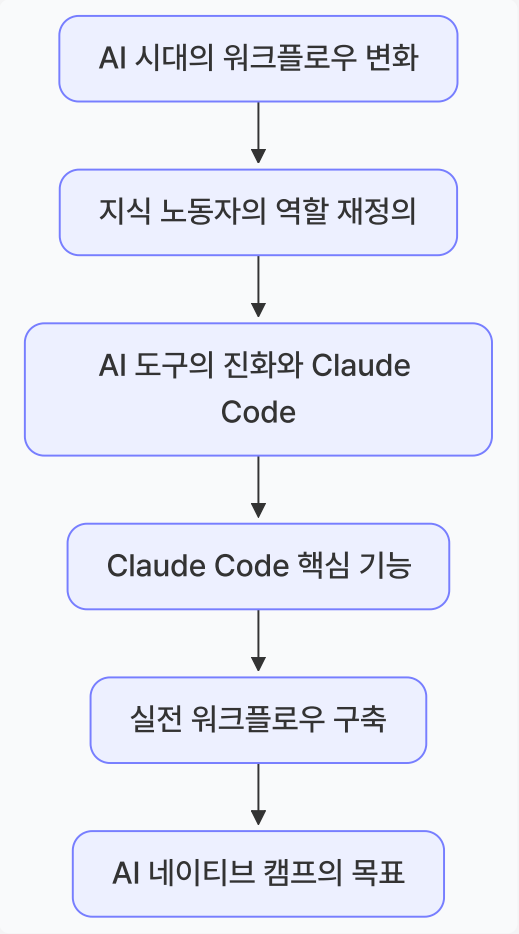
1. 개요
   1.1 서론
   1.2 전체 구조
2. AI 시대, 지식 노동의 본질과 워크플로우의 변화
   2.1 지식 노동자의 새로운 역할: '무엇을 시킬 것인가'
   2.2 도구의 진화: 추상화 계층의 상승과 진입 장벽 하락
   2.3 AI 네이티브 워크플로우: Claude Code를 통한 실전 적용
3. Claude Code의 핵심 기능과 활용 전략
   3.1 CLAUDE.md: AI의 영구 기억과 규칙서
   3.2 Skill: 반복 업무를 위한 맞춤형 레시피
   3.3 MCP (Model Context Protocol): 외부 도구 연결의 표준
   3.4 Subagent: 독립된 공간에서 전담 처리
   3.5 Agent Teams: 여러 AI 에이전트의 협업
   3.6 Hook: 특정 이벤트 시 자동 실행
   3.7 Plugin: 모든 기능을 묶어서 팀에 공유
4. Claude Code 환경 설정 및 CLI 기초
   4.1 Claude Code 설치 및 첫 대화
   4.2 CLI (Command Line Interface) 기초 및 터미널 활용
   4.3 Git과 GitHub 기초: 버전 관리와 협업
5. AI 네이티브 캠프의 목표와 비전
   5.1 캠프 참여자들의 니즈와 기대
   5.2 캠프의 7일 여정과 최종 목표

1. 개요

1.1 서론

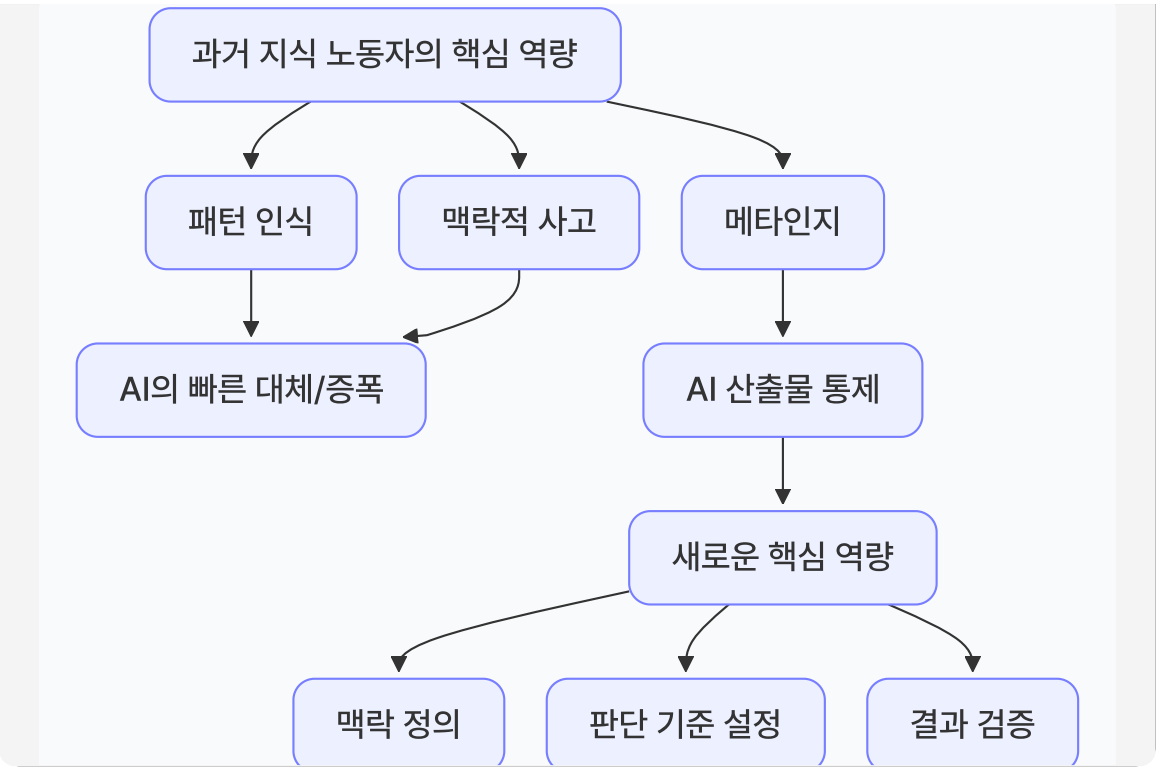
AI 시대의 도래는 우리가 일하는 방식에 근본적인 변화를 가져오고 있습니다. 특히 지식 노동의 본질이 변화하고 AI 도구가 진화하면서, 단순히 AI를 사용하는 것을 넘어 AI와 함께 일하는 새로운 워크플로우를 구축하는 것이 중요해졌습니다. 이 보고서는 AI 네이티브 캠프의 첫째 날 자료를 바탕으로, AI 시대의 워크플로우 변화와 Claude Code의 핵심 기능들을 깊이 있게 탐구하여 여러분이 AI를 효과적으로 활용하고 성장할 수 있도록 돕고자 합니다.

1.2 전체 구조



2. AI 시대, 지식 노동의 본질과 워크플로우의 변화

2.1 지식 노동자의 새로운 역할: '무엇을 시킬 것인가'



- 과거 지식 노동자의 핵심 역량은 **패턴 인식, 맥락적 사고, 메타인지**였습니다. [C]
  - AI는 방대한 데이터를 기반으로 패턴 인식과 정보 압축(200,000:1) 능력을 인간보다 빠르고 정확하게 수행하며, 맥락적 사고 능력도 빠르게 따라오고 있습니다. [C]
  - 이에 따라 지식 노동자의 역할은 단순히 정보를 처리하는 것을 넘어, AI가 무엇을 할지 정의하고, 판단 기준을 설정하며, 결과를 검증하는 메타인지적 능력으로 변화하고 있습니다. [C]
- 이는 마치 산업 혁명이 육체 노동을 해소했듯이, AI가 지식 노동의 많은 부분을 대체하거나 증폭시키면서 지식 노동의 본질 자체가 바뀌고 있음을 의미합니다. [C]

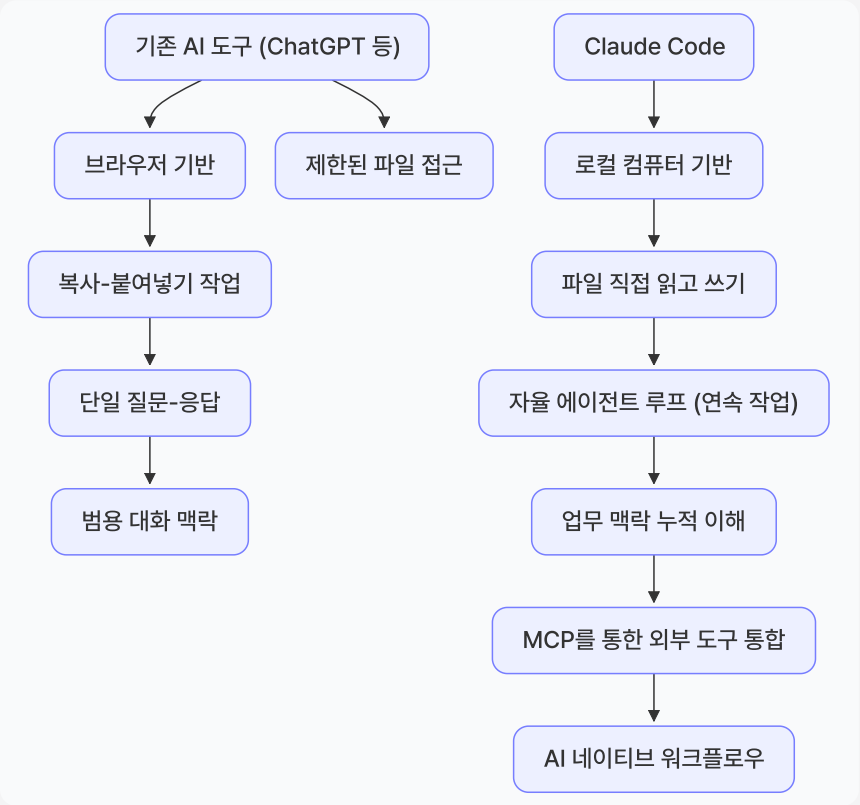
2.2 도구의 진화: 추상화 계층의 상승과 진입 장벽 하락

시대	주요 도구	추상화 계층	진입 장벽	특징
1950년대	기계어	낮음 (0과 1)	높음 (소수 전문가)	컴퓨터와 직접 대화, 복잡하고 어려움
1960년대	어셈블리어	중간 (기호)	중간	기호로 한 단계 추상화, 접근성 향상

1970년대	고급 언어	높음 (인간 논리)	낮음 (개발자)	인간의 논리에 가까운 문법, 대중화 시작
1990년대	API	매우 높음 (기능 단위)	매우 낮음 (비개발자도 활용)	기능 단위 호출, 소프트웨어 연동 용이
2010년대	SaaS	최고 (클릭)	거의 없음 (모든 사용자)	클릭만으로 소프트웨어 사용, 생산성 극대화
2020년대	자연어 (AI)	궁극 (대화)	없음 (생각할 수 있는 모든 사람)	말로 시키는 시대, AI가 직접 소프트웨어 제어

- 컴퓨터 도구는 기계어에서 시작하여 어셈블리어, 고급 언어, API, SaaS를 거쳐 자연어 기반의 AI 도구로 진화했습니다. [C]
  - 이러한 진화는 **추상화 계층의 지속적인 상승**을 의미하며, 이는 곧 컴퓨터의 힘을 활용할 수 있는 사람의 수가 기하급수적으로 늘어나는 결과를 가져왔습니다. [C]
  - 과거에는 개발자만이 가능했던 복잡한 작업들이 이제는 자연어 명령만으로도 가능해지면서, 모든 사람이 코딩을 통해 새로운 차원의 일을 할 수 있는 시대가 열리고 있습니다. [C]
- 특히 AI는 패턴 인식, 맥락적 사고, 메타인지 등 과거 지식 노동자의 핵심 역량을 빠르게 대체하거나 증폭시키고 있으며, 이는 인간의 노동력을 무한히 확장시키는 효과를 가져옵니다. [C]

2.3 AI 네이티브 워크플로우: Claude Code를 통한 실전 적용



- AI 네이티브 워크플로우는 AI를 단순한 도구가 아닌, 업무 환경의 핵심 주체로 활용하는 방식을 의미합니다. [C]
  - 기존 AI 도구(ChatGPT 등)는 주로 브라우저 기반으로 복사-붙여넣기, 단일 질문-응답 방식에 머물러 업무 맥락 이해에 한계가 있었습니다. [C]
  - 반면 **Claude Code**는 사용자의 로컬 컴퓨터에서 직접 파일을 읽고 쓰며, 연속된 작업을 자율적으로 수행하고 업무 맥락을 누적하여 이해하는 'AI 동료'와 같습니다. [C]
- Claude Code는 **자율 에이전트 루프**를 통해 사용자의 명령을 바탕으로 관련 파일 탐색, 패턴 분석, 초안 생성, 결과 검증 및 수정 등 여러 단계를 스스로 밟아 과업을 수행합니다. [C]
  - 이는 피터 드러커가 말한 지식 노동의 본질인 '자율적 과업 정의'를 AI에 적용하는 것으로, 사용자는 '무엇을 시킬지 정확히 아는 것'에 집중할 수 있게 됩니다. [C]
- 또한, **MCP(Model Context Protocol)**를 통해 Notion, Slack, GitHub 등 다양한 외부 업무 도구를 하나의 자연어 인터페이스로 통합 제어할 수 있어, 추상화 계층이 한 단계 더 올라간 효율적인 업무 환경을 구축합니다. [C]
- AI 네이티브 캠프는 이러한 Claude Code를 통해 도구 사용법을 넘어, 내 일의 맥락을 AI에게 주고, 판단 기준을 이식하며, 워크플로우를 재설계하는 경험을 제공하여 지식 노동의 새로운 형태를 체험하게 하는 것을 목표로 합니다. [C]

3. Claude Code의 핵심 기능과 활용 전략

3.1 CLAUDE.md: AI의 영구 기억과 규칙서

특징	설명	비유	활용 예시
근본 원리	AI가 대화를 시작할 때 가장 먼저 읽는 지시문 (시스템 프롬프트와 유사)	팀 위키, 영구 기억	"항상 존댓말로", "표로 정리해줘", "내 이름은 OOO"
작동 방식	세션 시작 시 CLAUDE.md를 항상 로딩하여 규칙을 아는 상태로 대화 시작	매번 읽는 규칙서	개인 정보, 선호하는 언어, 터미널 설정, 토큰 사용 전략 등
관리 방법	`/init` 명령어로 자동 생성 및 업데이트, 세션마다 업데이트 여부 결정	프로젝트의 의도와 관점을 AI에게 알려주는 역할	프로젝트 목적, 코드베이스 설명, 하위 폴더별 컨텍스트 정의

- **CLAUDE.md**는 Claude Code가 매번 대화를 시작할 때 가장 먼저 읽는 규칙서이자 영구 기억과 같습니다. [C]
  - 이는 AI에게 시스템 프롬프트처럼 작동하여, 대화의 기본 규칙, 사용자 정보, 선호하는 작업 방식 등을 미리 설정할 수 있게 합니다. [C]
  - 예를 들어, "항상 존댓말로 대화해줘", "내 이름은 정구봉이야", "파이썬을 사용할 때는 UV 패키지 관리자를 써줘"와 같은 지시를 CLAUDE.md에 넣어 AI가 일관된 방식으로 작업하도록 유도할 수 있습니다. [C]
- CLAUDE.md는 세션이 시작될 때마다 항상 로딩되므로, 변경 사항을 적용하려면 세션을 종료하고 다시 시작해야 합니다. [C]
  - `/init` 명령어를 통해 코드베이스를 읽어 CLAUDE.md를 자동으로 생성하거나 업데이트할 수 있으며, 이를 통해 프로젝트의 의도와 관점을 AI에게 명확히 전달하는 것이 중요합니다. [C]
- CLAUDE.md에 토큰 사용 전략(예: 토큰을 무조건 많이 사용)하여 AI의 작업 방식을 최적화할 수 있습니다. [C]
  - 이는 AI의 'IQ'를 높이는 것과 같아서, AI가 더 똑똑하게 일하도록 돕습니다. [C]

3.2 Skill: 반복 업무를 위한 맞춤형 레시피

특징	설명	비유	활용 예시
근본 원리	점진적 로딩 (필요할 때만 컨텍스트 로딩)	레시피 북	주간 보고서 자동 생성, 미팅 요약, 데이터 분석
작동 방식	CLAUDE.md와 달리, Skill은 호출될 때만 해당 컨텍스트를 읽음	컨텍스트 윈도우 효율적 사용	`/weekly-sync` 명령으로 Slack, Calendar, Notion 정보 수집 및 문서 생성
폴더 구조	`my-skill/SKILL.md` (필수), `scripts/`, `references/`, `assets/` (선택)	체계적인 레시피 관리	복잡한 다단계 작업 자동화, 특정 도메인 지식 활용

- **Skill**은 반복적인 업무를 레시피처럼 저장하여 한 줄 명령으로 실행할 수 있게 하는 기능입니다. [C]
  - CLAUDE.md가 항상 로딩되는 것과 달리, Skill은 필요할 때만 컨텍스트를 로딩하는 '점진적 로딩' 방식을 사용합니다. [C]
  - 이는 AI의 **컨텍스트 윈도우(Context Window)**를 효율적으로 사용하여, AI의 성능을 최적화하고 다양한 작업을 동시에 수행할 수 있게 합니다. [C]
- Skill은 `SKILL.md` 파일을 중심으로 `scripts/`, `references/`, `assets/` 등의 폴더 구조를 가질 수 있어, 복잡한 작업도 체계적으로 관리할 수 있습니다. [C]
  - 예를 들어, `/weekly-sync` Skill은 Slack, Calendar, Notion 등 여러 소스에서 정보를 수집하고, Git을 확인하며, 주간 보고서 문서를 자동으로 생성하는 다단계 작업을 수행할 수 있습니다. [C]
- Skill은 컨텍스트를 적게 사용하므로, 수많은 Skill을 만들어 저장해두고 필요할 때마다 불러와 사용할 수 있습니다. [C]
  - Skill의 `name`과 `description` 정보만 초기 로딩 시 사용되며, 실제 내용은 Skill이 호출될 때 읽힙니다. [C]

3.3 MCP (Model Context Protocol): 외부 도구 연결의 표준

특징	설명	비유	활용 예시
근본 원리	툴 콜링 (Tool Calling)	외부 서비스와 통신하는 표준 규격	Slack, Notion, GitHub, Google Calendar 등 연결
작동 방식	AI가 텍스트 생성 외에 외부 함수를 호출하여 소프트웨어 직접 제어	AI 에이전트의 팔다리	"지난주 회의 내용 정리해서 Linear에 이슈로 만들고, 담당자에게 Slack으로 알려줘"
장점	하나의 자연어 인터페이스로 여러 업무 도구 통합 제어, 추상화 계층 상승	AI가 컴퓨터의 모든 일을 할 수 있게 함	200명 지원서 분석, 미팅 녹취록 자동 이슈 생성, CRM 데이터 리포트 자동화

- **MCP(Model Context Protocol)**는 AI가 텍스트 생성뿐만 아니라 "이 함수를 이 파라미터로 호출해"와 같이 외부 도구를 직접 제어할 수 있게 하는 오픈 표준 프로토콜입니다. [C]
  - 이는 AI 에이전트가 소프트웨어를 직접 제어할 수 있는 '툴 콜링' 능력을 외부 서비스까지 확장하는 개념입니다. [C]
  - MCP를 통해 Claude Code는 Notion, Linear, Slack, GitHub, Google Calendar 등 다양한 외부 업무 도구와 연결되어 하나의 자연어 인터페이스로 통합 제어할 수 있습니다. [C]
- MCP는 추상화 계층을 한 단계 더 상승시켜, 사용자가 복잡한 코딩 없이도 AI에게 "지난주 회의 내용 정리해서 Linear에 이슈로 만들고, 담당자한테 Slack으로 알려줘"와 같은 복합적인 명령을 내릴 수 있게 합니다. [C]
  - 이는 AI가 컴퓨터로 할 수 있는 모든 종류의 일을 수행할 수 있도록 권한을 부여하는 것과 같으며, 비개발자도 AI를 통해 200명 지원서 분석, 미팅 녹취록 자동 이슈 생성, CRM 데이터 리포트 자동화 등 다양한 업무를 직접 수행할 수 있게 됩니다. [C]
- MCP는 로컬 컴퓨터의 모든 데이터와 환경을 연결하여 AI가 통제할 수 있게 함으로써, 기존에 개발자에게 의존해야 했던 많은 작업들을 사용자가 직접 수행할 수 있도록 합니다. [C]

3.4 Subagent: 독립된 공간에서 전담 처리

특징	설명	비유	활용 예시
근본 원리	프로세스 격리 + Blank Slate	부하 직원 (1:1 위임, 보고만)	주간 싱크 (캘린더, 슬랙, 메일 등 병렬 수집), 리서치, 코드 보안 분석
작동 방식	메인 대화의 컨텍스트를 물려받지 않고 빈 상태에서 작업 후 요약만 반환	컨텍스트 윈도우 초기화, 토큰 효율적 사용	여러 소스에서 정보 병렬 수집, 메인 에이전트의 컨텍스트 오염 방지
장점	컨텍스트 오염 방지, 병렬 작업 가능, 시간 효율성 증대	독립적인 작업 수행, 메인 에이전트의 부담 경감	프론트엔드/백엔드 동시 변경 시 충돌 가능성 유의

- **Subagent**는 메인 에이전트의 컨텍스트를 물려받지 않고 독립된 공간에서 빈 상태(Blank Slate)로 작업을 전담 처리하는 기능입니다. [C]
  - 이는 메인 에이전트의 컨텍스트 윈도우가 꽉 차는 것을 방지하고, 토큰을 효율적으로 사용하여 AI의 작업 효율성을 높입니다. [C]
  - Subagent는 작업을 마친 후 결과를 요약하여 메인 에이전트에게 보고하며, 이때 Subagent가 사용한 토큰은 메인 에이전트의 컨텍스트에 포함되지 않습니다. [C,A]
- Subagent는 병렬 작업에 매우 유용합니다. 예를 들어, 주간 싱크 작업 시 캘린더, 슬랙, 메일 등 여러 소스에서 정보를 동시에 수집하는 데 활용될 수 있습니다. [C,A]
  - 특히 리서치와 같이 '읽기(Read)' 토큰 사용량이 압도적으로 많은 태스크에 Subagent를 병렬로 돌리는 것이 효과적입니다. [A]
- 하지만 '쓰기(Write)' 토큰 사용량이 많은 태스크(예: 프론트엔드와 백엔드 코드 동시 변경)해야 합니다. [A]
- Subagent는 메인 에이전트의 컨텍스트를 물려받아 시작할 수도 있고, 완전히 빈 상태에서 시작할 수도 있는 등 다양한 옵션을 통해 유연하게 활용할 수 있습니다. [A]

3.5 Agent Teams: 여러 AI 에이전트의 협업

특징	설명	비유	활용 예시
근본 원리	여러 Agent가 동시에 협업, 팀원 간 소통 및 공유 태스크	프로젝트 팀 (팀원끼리 소통 + 공유 태스크)	UX 분석, 기술 아키텍처 설계, 데빌스 애드버킷 역할 수행
Subagent와의 차이	Subagent는 1:1 위임 및 보고만, Teams는 팀원 간 직접 메시지 전송 및 협업	고차원적인 협업, 중간 피드백 가능	복잡한 프로젝트의 다각적 분석 및 해결
작동 방식	메인 에이전트가 여러 팀을 호출, 팀원 간 통신하며 공유 작업 목록 완료	새로운 터미널 창에서 독립적으로 실행, 중간 제어 가능	UX 애널리스트, 기술 아키텍트, 데빌스 애드버킷 등 역할 분담

- **Agent Teams**는 여러 AI 에이전트가 동시에 협업하며, 팀원끼리 직접 메시지를 주고받고 공유 태스크를 완료하는 기능입니다. [C,A]
  - Subagent가 메인 에이전트에게 1:1로 위임하고 보고하는 '부하 직원'과 같다면, Agent Teams는 팀원 간 소통하며 프로젝트를 함께 진행하는 '프로젝트 팀'에 비유할 수 있습니다. [C,A]
  - Agent Teams는 새로운 터미널 창이 열려 독립적으로 실행되며, 메인 에이전트는 팀원들의 작업을 기다리면서도 중간에 개입하여 명령을 내리거나 작업을 취소할 수 있습니다. [C,B]
- 이 기능은 복잡한 프로젝트를 다각적으로 분석하고 해결하는 데 유용합니다. 예를 들어, UX 애널리스트, 기술 아키텍트, 데빌스 애드버킷 등 여러 관점을 가진 팀을 구성하여 CLI 도구 개발 프로젝트를 진행할 수 있습니다. [B]
- Agent Teams는 Subagent보다 더 많은 토큰을 사용하며, 현재는 리서치 프리뷰 기능으로 제공됩니다. [B]
  - 모든 작업이 완료되면 자동으로 창이 닫히므로, 지속적인 상호작용을 기대하는 용도보다는 특정 태스크의 고차원적인 협업에 적합합니다. [B]

3.6 Hook: 특정 이벤트 시 자동 실행

특징	설명	비유	활용 예시
근본 원리	결정론적 프로그래밍	100% 확실하게 동작하는 자동화 스크립트	파일 저장 후 코드 포맷팅, 세션 종료 시간 기록, AI 응답 후 요약 음성 출력
AI와의 차이	AI는 확률적(대부분 맞지만 가끔 까먹음), Hook은 코드 실행으로 100% 확실	신뢰성 높은 자동화	코드 품질 관리, 작업 흐름의 특정 지점 자동화
활용 분야	개발 업무 (코드 포맷팅, 린팅), 개인 생산성 (세션 관리, 알림)	개발자에게 특히 유용, 비개발자도 특정 자동화에 활용 가능	AI 응답 후 10글자 이내로 요약하여 음성 출력

- **Hook**은 특정 이벤트(예: 파일 저장, AI 응답 완료) 발생 시 미리 정의된 코드를 100% 확실하게 자동 실행하는 기능입니다. [C,B]
  - AI가 "대부분" 맞지만 가끔 까먹을 수 있는 '확률적' 방식과 달리, Hook은 코드가 실행되므로 100% 확실하게 동작합니다. [C]
  - 이는 결정론적 프로그래밍 원리를 기반으로 하여, 신뢰성 높은 자동화를 구현할 수 있게 합니다. [C]
- Hook은 주로 개발 업무에서 코드 저장 후 자동으로 포맷팅(prettier --write.)하거나 린팅(linting)을 실행하는 데 사용됩니다. [C,B]
  - 개인 생산성 측면에서는 AI 세션 종료 시간을 기록하거나, AI 응답이 나온 후 이를 짧게 요약하여 음성으로 출력하는 등의 자동화를 구현할 수 있습니다. [B]
- 비개발자에게는 개발 업무만큼 효용이 크지 않을 수 있지만, 특정 작업 흐름의 신뢰성 높은 자동화가 필요할 때 유용하게 활용될 수 있습니다. [B]

3.7 Plugin: 모든 기능을 묶어서 팀에 공유

특징	설명	비유	활용 예시

근본 원리	패키징과 배포	모든 기능을 묶어서 팀에 공유하는 설치 단위	마케팅 플러그인 (Skill, MCP, Hook, Agent 등 포함)
장점	팀 전체의 동일한 환경 구축, 기능 재사용 및 배포 용이	효율적인 협업, 표준화된 업무 환경 제공	특정 도메인 지식, 업무 자동화 템플릿 공유
스코프	User Scope (어디서든 사용), Project Scope (특정 프로젝트에서 사용), Local Scope (현재 폴더에서만 사용)	유연한 설치 및 관리	개인 설정, 프로젝트별 맞춤 기능, 임시 기능

- **Plugin**은 Skill, MCP, Hook, Agent 등 다양한 Claude Code의 기능들을 하나의 설치 단위로 묶어서 팀 전체에 공유할 수 있게 하는 기능입니다. [C,B]
  - 이는 개별 기능을 수동으로 복사하거나 설정할 필요 없이, 팀 전체가 동일한 업무 환경을 갖추고 효율적으로 협업할 수 있도록 돕습니다. [C]
  - Plugin은 마치 MCP처럼 누군가 잘 만들어 놓은 기능을 가져다 쓰는 개념으로, 기능의 재사용성과 배포 용이성을 높입니다. [B]
- Plugin은 설치 시 세 가지 스코프(Scope)를 가질 수 있습니다. [B]
  - **User Scope**: 사용자가 어디서 Claude Code를 실행하든 항상 활성화되는 플러그인 (개인 설정) [B]
  - **Project Scope**: 특정 프로젝트 폴더 내에서만 활성화되는 플러그인 (프로젝트별 맞춤 기능) [B]
  - **Local Scope**: 현재 작업 중인 폴더에서만 활성화되는 플러그인 (임시 또는 특정 작업용) [B]
- 플러그인은 `claude plugin add` 명령어를 통해 설치할 수 있으며, 마켓플레이스에서 다양한 플러그인을 찾아 활용할 수 있습니다. [C,B]
  - 개발자가 아닌 경우, 다른 사람이 잘 만들어 놓은 플러그인을 가져다 쓰는 것이 매우 유용합니다. [B]

4. Claude Code 환경 설정 및 CLI 기초

4.1 Claude Code 설치 및 첫 대화

단계	설명	명령어/설정	비고
2. 첫 실행	터미널에 `claude` 입력하여 실행	`claude`	Claude 구독 필요 (최소 $20/월)
3. Output Style 설정	AI의 응답 스타일을 '설명형'으로 변경	`/output-style` → `Explanatory` 선택	AI와의 소통에 익숙하지 않을 때 유용
4. 자유 대화	AI에게 자신을 소개하고 자유롭게 대화 시작	"안녕, 나는 [이름]이고 [직업]이야"	AI의 맥락 이해 능력 확인

- Claude Code는 Mac/Linux 또는 Windows PowerShell에서 간단한 한 줄 명령어로 설치할 수 있으며, Node.js가 필요 없고 자동 업데이트 기능이 내장되어 있습니다. [C]
  - 설치 후 터미널에 `claude`를 입력하여 실행하며, Claude AI 구독(최소 $20/월)이 필수입니다. [C]
- AI와의 첫 대화 시, `/output-style` 명령어를 통해 응답 스타일을 'Explanatory(설명형)'로 설정하면 AI가 더 자세하고 친절하게 설명해 주어 초보자에게 유용합니다. [C]
  - 이 설정은 한 번 해두면 계속 유지되며, `Default`로 변경하여 되돌릴 수 있습니다. [C]
- Claude Code는 데스크톱 앱으로도 사용 가능하지만, 터미널(CLI) 환경에서 사용하는 것이 새로운 기능에 대한 빠른 접근성, 병렬 작업 효율성, 컴퓨터의 모든 기능 제어 측면에서 훨씬 유리합니다. [C,A]

4.2 CLI (Command Line Interface) 기초 및 터미널 활용

명령어	의미	비유	활용 예시 (Claude Code)
`cd [폴더명]`	폴더 이동 (Change Directory)	파인더에서 폴더 더블클릭	`!cd my-project` (느낌표로 직접 CLI

			명령어 실행)
`ls`	현재 폴더 목록 보기 (List)	폴더 열어서 내용 확인	`!ls` (현재 폴더 파일 목록 확인)
`pwd`	현재 위치 확인 (Print Working Directory)	주소창 보기	`!pwd` (현재 작업 디렉토리 확인)
`cat [파일명]`	파일 내용 보기 (Concatenate)	메모장으로 파일 열어보기	`!cat README.md` (README.md 파일 내용 출력)
`open [파일명/폴더명]`	파일/폴더 열기	파인더에서 더블클릭	`!open.` (현재 폴더를 파인더로 열기)

- **CLI(Command Line Interface)**는 명령어를 통해 컴퓨터와 상호작용하는 방식으로, GUI(Graphic User Interface)와 달리 컴퓨터의 모든 기능을 자유롭게 제어할 수 있습니다. [C]
  - Claude Code는 CLI 환경에서 실행되므로, 컴퓨터의 모든 파일에 직접 접근하고 수정할 수 있습니다. [C]
- CLI 기초 명령어 5가지(`cd`, `ls`, `pwd`, `cat`, `open`)만 알아도 Claude Code를 활용하는 데 충분합니다. [C]
  - Claude Code는 명령어를 몰라도 자연어로 지시하면 AI가 알아서 처리해 줍니다. [C]
  - 특정 CLI 명령어를 직접 실행하고 싶을 때는 명령어 앞에 `!` (느낌표)를 붙여 사용합니다. [B]
- 터미널 프로그램으로는 Ghostty(빠르고 가벼움, GPU 가속, 예쁜 UI)합니다. [C,B]
  - 터미널 창을 분할하여 여러 작업을 동시에 수행하려면 `Command + D` (세로 분할) 또는 `Command + Shift + D` (가로 분할)를 사용할 수 있습니다. [B]
- VSCode, Cursor, Antigravity와 같은 코드 에디터(IDE)하면, Claude Code가 생성한 파일들을 시각적으로 쉽게 확인하고 관리할 수 있습니다. [C]

4.3 Git과 GitHub 기초: 버전 관리와 협업

개념	의미	비유	활용 예시 (Claude Code)

Git	코드 버전 관리 시스템	게임 세이브, 실행 취소의 끝판왕	`git commit` (작업 내용 저장), `git branch` (A안, B안 동시 작업)
GitHub	Git 저장소를 호스팅하는 웹 서비스	Google Drive에 폴더 업로드	`git clone` (원격 저장소 다운로드), `git push` (변경 사항 업로드)
Pull Request (PR)	변경 사항을 메인 코드에 반영하기 위한 검토 요청	"이렇게 바꿨는데 괜찮아?" 리뷰 요청	AI가 코드 변경 후 PR 생성 및 리뷰 요청
Merge	PR이 승인된 후 변경 사항을 메인 코드에 병합	"오케이, 합치자" 리뷰 통과 후 반영	AI가 Merge 작업 자동화

- **Git**은 코드의 변경 이력을 관리하고 여러 사람이 협업할 수 있도록 돕는 버전 관리 시스템입니다. [C]
  - `git commit`은 작업 내용을 저장하는 '게임 세이브'와 같고, `git branch`는 'A안, B안 동시 작업'처럼 여러 갈래로 작업을 진행할 수 있게 합니다. [C,B]
- **GitHub**는 Git 저장소를 호스팅하는 웹 서비스로, 원격 저장소에 코드를 업로드하고 공유하는 데 사용됩니다. [C]
  - `git clone`은 원격 저장소의 폴더를 로컬로 다운로드하는 것과 같고, `git push`는 로컬의 변경 사항을 원격 저장소에 업로드하는 것입니다. [C,B]
- **Pull Request(PR)**는 자신이 변경한 코드를 메인 코드에 반영하기 전에 다른 사람에게 검토를 요청하는 과정입니다. [C]
  - Claude Code는 이러한 Git 및 GitHub 작업을 자연어 명령으로 대신 수행할 수 있으므로, 사용자는 개념만 이해하고 있어도 AI를 통해 효율적으로 버전 관리를 할 수 있습니다. [B]
- Git과 GitHub 지식은 Claude Code를 활용하여 복잡한 코드 개발 및 협업 업무를 수행하는 데 필수적입니다. [B]

5. AI 네이티브 캠프의 목표와 비전

5.1 캠프 참여자들의 니즈와 기대

유형	비율	주요 니즈	예시 업무
Type A	42%	반복 업무 시스템화	VoC → PRD → 기획서, 미팅 녹취 → CRM → 팔로우업 메일
Type B	22%	직접 만들 수 있는 사람	도메인 지식 → 프로토타입, 계약서 기반 Agentic Workflow
Type C	20%	조직 변화 방향 찾기	AX TF 구축 선행 학습, CEO의 Enterprise adoption ambassador
Type D	16%	AI 활용 방향 설정	PO의 시대 변화 불안감 해소, AI 개발자의 AI-native 업무 방식 탐색

- AI 네이티브 캠프 참가자들은 다양한 배경과 니즈를 가지고 있으며, 크게 네 가지 유형으로 분류됩니다. [C]
  - **Type A (42%)**: 반복 업무를 시스템으로 자동화하고 싶어 하는 사람들로, 구체적인 자동화 대상을 이미 알고 있습니다. [C]
  - **Type B (22%)**: 도메인 지식을 활용하여 직접 프로토타입을 만들고 싶어 하는 사람들입니다. [C]
  - **Type C (20%)**: 조직을 AI 네이티브하게 변화시키고 싶어 하는 리더 포지션의 사람들입니다. [C]
  - **Type D (16%)**: AI를 활용해야 한다는 것은 알지만, 어디서부터 시작해야 할지 방향을 잡고 싶어 하는 사람들입니다. [C]
- 참가자 중 65%가 의사결정권자(PM/PO, CEO/창업자 등)이며, 다양한 산업 분야에서 AI를 이미 사용하고 있지만 Claude Code 경험자는 적어, 더 깊은 활용에 대한 갈증을 가지고 있습니다. [C]

5.2 캠프의 7일 여정과 최종 목표

Day	주요 학습 내용	기대 효과

Day 1	"이게 되네?" 첫 경험 (Claude Code 설치 및 프레임워크 익히기)	AI 네이티브 업무 환경에 대한 전체적인 관점 형성
Day 2	AI에게 내 일을 설명하는 법 (CLAUDE.md, Skill 활용)	AI가 내 맥락을 이해하게 만들기
Day 3	내 도구들을 연결하는 법 (MCP 활용)	AI와 함께 일하는 시스템 구축
Day 4	반복 업무를 시스템으로 (Subagent, Agent Teams 활용)	내 일에 작동하는 자동화 구축
Day 5	도구가 바뀌어도 흔들리지 않는 원리 (Hook, Plugin 활용)	AI 도구 변화에 유연하게 대처하는 능력
Day 6	실전 적용 + 크로스 피드백 (PRD 기획서 작성)	50명의 시행착오에서 얻은 인사이트
Day 7	"내 일이 이렇게 바뀌었다" (원하는 시스템 완성)	혼자서도 계속할 수 있는 원리 체득

- AI 네이티브 캠프는 7일간의 밀도 높은 여정을 통해 참가자들이 AI를 활용한 새로운 워크플로우를 직접 경험하고 체득하도록 설계되었습니다. [C]
  - 캠프는 단순히 도구 사용법을 가르치는 것이 아니라, AI 네이티브한 업무 환경에 대한 전체적인 프레임워크와 관점을 형성하고, 스스로의 워크플로우에 대한 통제권을 갖도록 돕는 것을 목표로 합니다. [C]
  - 참가자들은 AI에게 자신의 일을 설명하는 법, 도구들을 연결하는 법, 반복 업무를 시스템화하는 법 등을 배우며, 궁극적으로는 도구가 바뀌어도 흔들리지 않는 AI 활용 원리를 익히게 됩니다. [C]
- 캠프의 최종 목표는 참가자들이 자신의 일에 작동하는 자동화를 구축하고, 50명의 동료들과의 협업을 통해 혼자서는 3개월 걸릴 일을 1주일 만에 압축적으로 경험하며, 혼자서도 AI 활용을 계속할 수 있는 원리를 갖추는 것입니다. [C]
  - 이는 "일하는 방식이 바뀌면, AI 도구는 자연스럽게 따라온다"는 캠프의 핵심 철학을 반영합니다. [C]
- 캠프 기간 동안 참가자들은 Slack 채널을 통해 서로의 시행착오와 꿀팁을 공유하며 집단 지성을 구축하고, AI에게 직접 질문하여 학습하는 방식을 적극적으로 활용하게 됩니다. [C,B]